

-
- Loops
 - The *while* Loop
 - Tips and Traps
 - More Operators
 - The *for* Loop
 - Nesting of Loops
 - Multiple Initialisations in the *for* Loop
 - The Odd Loop
 - The *break* Statement
 - The *continue* Statement
 - The *do-while* Loop
 - Summary
 - Exercise

The programs that we have developed so far used either a sequential or a decision control instruction. In the first one, the calculations were carried out in a fixed order, while in the second, an appropriate set of instructions were executed depending upon the outcome of the condition being tested (or a logical decision being taken).

These programs were of limited nature, because when executed, they always performed the same series of actions, in the same way, exactly once. Almost always, if something is worth doing, it's worth doing more than once. You can probably think of several examples of this from real life, such as eating a good dinner or going for a movie. Programming is the same; we frequently need to perform an action over and over, often with variations in the details each time. The mechanism, which meets this need, is the 'loop', and loops are the subject of this chapter.

Loops

The versatility of the computer lies in its ability to perform a set of instructions repeatedly. This involves repeating some portion of the program either a specified number of times or until a particular condition is being satisfied. This repetitive operation is done through a loop control instruction.

There are three methods by way of which we can repeat a part of a program. They are:

- (a) Using a **for** statement
- (b) Using a **while** statement
- (c) Using a **do-while** statement

Each of these methods is discussed in the following pages.

The *while* Loop

It is often the case in programming that you want to do something a fixed number of times. Perhaps you want to calculate gross salaries of ten different persons, or you want to convert temperatures from centigrade to fahrenheit for 15 different cities.

The **while** loop is ideally suited for such cases. Let us look at a simple example, which uses a **while** loop. The flowchart shown below would help you to understand the operation of the **while** loop.

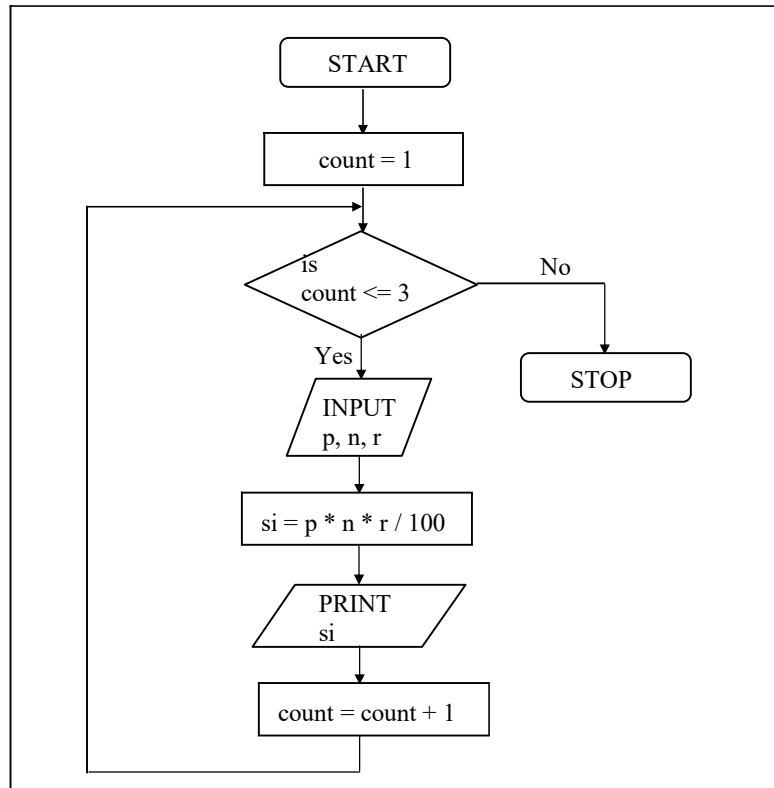


Figure 3.1

```
/* Calculation of simple interest for 3 sets of p, n and r */  
main( )  
{  
    int p, n, count;  
    float r, si;  
  
    count = 1;
```

```
while ( count <= 3 )
{
    printf ( "\nEnter values of p, n and r " );
    scanf ( "%d %d %f", &p, &n, &r );
    si = p * n * r / 100 ;
    printf ( "Simple interest = Rs. %f", si );

    count = count + 1 ;
}
```

And here are a few sample runs...

```
Enter values of p, n and r 1000 5 13.5
Simple interest = Rs. 675.000000
Enter values of p, n and r 2000 5 13.5
Simple interest = Rs. 1350.000000
Enter values of p, n and r 3500 5 3.5
Simple interest = Rs. 612.500000
```

The program executes all statements after the **while** 3 times. The logic for calculating the simple interest is written within a pair of braces immediately after the **while** keyword. These statements form what is called the ‘body’ of the **while** loop. The parentheses after the **while** contain a condition. So long as this condition remains true all statements within the body of the **while** loop keep getting executed repeatedly. To begin with the variable **count** is initialized to 1 and every time the simple interest logic is executed the value of **count** is incremented by one. The variable **count** is many a times called either a ‘loop counter’ or an ‘index variable’.

The operation of the **while** loop is illustrated in the following figure.

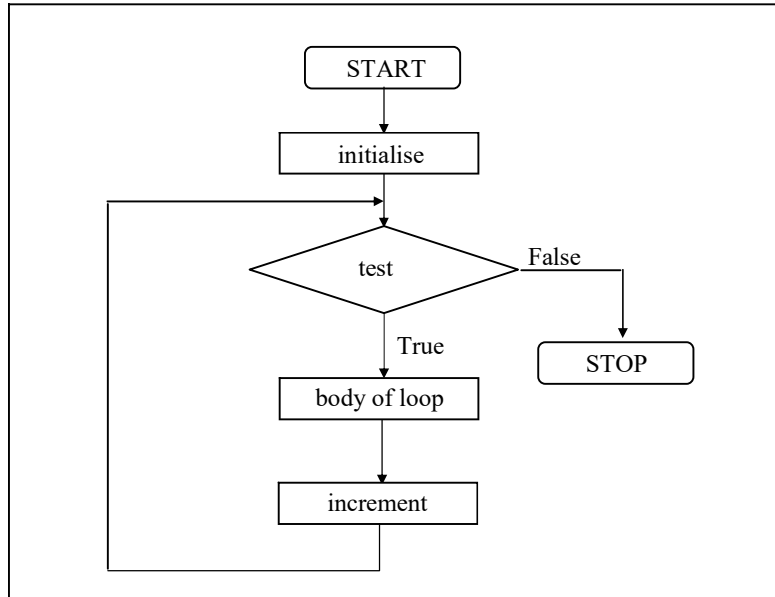


Figure 3.2

Tips and Traps

The general form of **while** is as shown below:

```
initialise loop counter ;  
while ( test loop counter using a condition )  
{  
    do this ;  
    and this ;  
    increment loop counter ;  
}
```

Note the following points about **while...**

- The statements within the **while** loop would keep on getting executed till the condition being tested remains true. When the

condition becomes false, the control passes to the first statement that follows the body of the **while** loop.

In place of the condition there can be any other valid expression. So long as the expression evaluates to a non-zero value the statements within the loop would get executed.

- The condition being tested may use relational or logical operators as shown in the following examples:

```
while ( i <= 10 )
while ( i >= 10 && j <= 15 )
while ( j > 10 && ( b < 15 || c < 20 ) )
```

- The statements within the loop may be a single line or a block of statements. In the first case the parentheses are optional. For example,

```
while ( i <= 10 )
    i = i + 1 ;
```

is same as

```
while ( i <= 10 )
{
    i = i + 1 ;
}
```

- As a rule the while must test a condition that will eventually become false, otherwise the loop would be executed forever, indefinitely.

```
main( )
{
    int i = 1 ;
    while ( i <= 10 )
        printf ( "%d\n", i ) ;
}
```

This is an indefinite loop, since `i` remains equal to 1 forever. The correct form would be as under:

```
main( )
{
    int i = 1 ;
    while ( i <= 10 )
    {
        printf ( "%d\n", i ) ;
        i = i + 1 ;
    }
}
```

- Instead of incrementing a loop counter, we can even decrement it and still manage to get the body of the loop executed repeatedly. This is shown below:

```
main( )
{
    int i = 5 ;
    while ( i >= 1 )
    {
        printf ( "\nMake the computer literate!" ) ;
        i = i - 1 ;
    }
}
```

- It is not necessary that a loop counter must only be an int. It can even be a float.

```
main( )
{
    float a = 10.0 ;
    while ( a <= 10.5 )
    {
        printf ( "\nRaindrops on roses..." ) ;
        printf ( "...and whiskers on kittens" ) ;
        a = a + 0.1 ;
    }
}
```

```
    }  
}
```

- Even floating point loop counters can be decremented. Once again the increment and decrement could be by any value, not necessarily 1.

What do you think would be the output of the following program?

```
main()  
{  
    int i = 1;  
    while ( i <= 32767 )  
    {  
        printf ( "%d\n", i );  
        i = i + 1;  
    }  
}
```

No, it doesn't print numbers from 1 to 32767. It's an indefinite loop. To begin with, it prints out numbers from 1 to 32767. After that value of **i** is incremented by 1, therefore it tries to become 32768, which falls outside the valid integer range, so it goes to other side and becomes -32768 which would certainly satisfy the condition in the **while**. This process goes on indefinitely.

- What will be the output of the following program?

```
main()  
{  
    int i = 1;  
    while ( i <= 10 );  
    {  
        printf ( "%d\n", i );  
        i = i + 1;  
    }  
}
```

This is another indefinite loop, and it doesn't give any output at all. The reason is, we have carelessly given a `;` after the **while**. This would make the loop work like this...

```
while ( i <= 10 )
;
{
    printf ( "%d\n", i );
    i = i + 1 ;
}
```

Since the value of `i` is not getting incremented the control would keep rotating within the loop, eternally. Note that enclosing **printf ()** and **i = i + 1** within a pair of braces is not an error. In fact we can put a pair of braces around any individual statement or set of statements without affecting the execution of the program.

More Operators

There are variety of operators which are frequently used with **while**. To illustrate their usage let us consider a problem wherein numbers from 1 to 10 are to be printed on the screen. The program for performing this task can be written using **while** in the following different ways:

```
(a) main( )
{
    int i = 1 ;
    while ( i <= 10 )
    {
        printf ( "%d\n", i );
        i = i + 1 ;
    }
}
```

```
(b) main( )
{
    int i = 1 ;
    while ( i <= 10 )
    {
        printf ( "%d\n", i ) ;
        i++ ;
    }
}
```

Note that the increment operator `++` increments the value of `i` by 1, every time the statement `i++` gets executed. Similarly, to reduce the value of a variable by 1 a decrement operator `--` is also available.

However, never use `n+++` to increment the value of `n` by 2, since C doesn't recognize the operator `+++`.

```
(c) main( )
{
    int i = 1 ;
    while ( i <= 10 )
    {
        printf ( "%d\n", i ) ;
        i += 1 ;
    }
}
```

Note that `+=` is a compound assignment operator. It increments the value of `i` by 1. Similarly, `j = j + 10` can also be written as `j += 10`. Other compound assignment operators are `-=`, `*=`, `/=` and `%=`.

```
(d) main( )
{
    int i = 0 ;
    while ( i++ < 10 )
```

```
        printf ( "%d\n", i );  
    }
```

In the statement **while (i++ < 10)**, firstly the comparison of value of **i** with 10 is performed, and then the incrementation of **i** takes place. Since the incrementation of **i** happens after its usage, here the **++** operator is called a post-incrementation operator. When the control reaches **printf()**, **i** has already been incremented, hence **i** must be initialized to 0.

```
(e) main( )  
{  
    int i = 0 ;  
    while ( ++i <= 10 )  
        printf ( "%d\n", i );  
}
```

In the statement **while (++i <= 10)**, firstly incrementation of **i** takes place, then the comparison of value of **i** with 10 is performed. Since the incrementation of **i** happens before its usage, here the **++** operator is called a pre-incrementation operator.

The *for* Loop

Perhaps one reason why few programmers use **while** is that they are too busy using the **for**, which is probably the most popular looping instruction. The **for** allows us to specify three things about a loop in a single line:

- (a) Setting a loop counter to an initial value.
- (b) Testing the loop counter to determine whether its value has reached the number of repetitions desired.
- (c) Increasing the value of loop counter each time the program segment within the loop has been executed.

The general form of **for** statement is as under:

```
for ( initialise counter ; test counter ; increment counter )
{
    do this ;
    and this ;
    and this ;
}
```

Let us write down the simple interest program using **for**. Compare this program with the one, which we wrote using **while**. The flowchart is also given below for a better understanding.

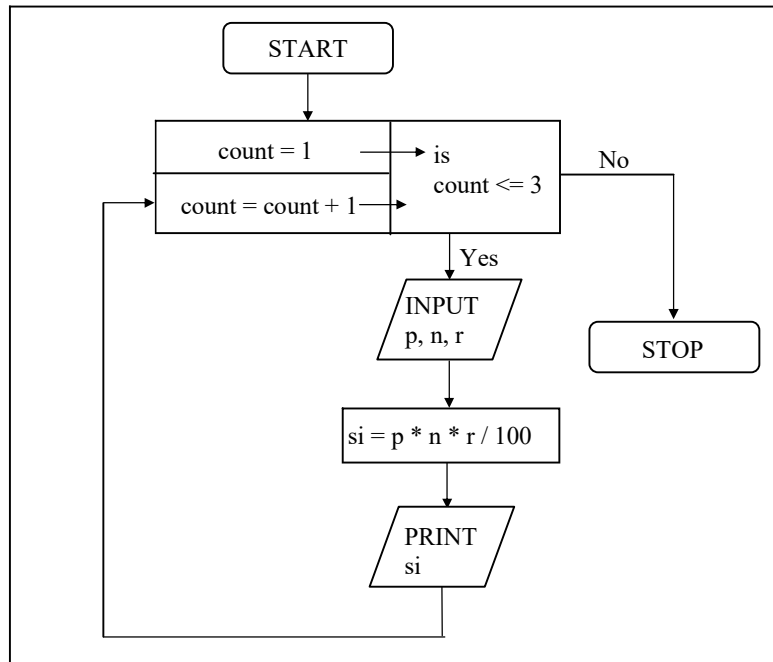


Figure 3.3

```

/* Calculation of simple interest for 3 sets of p, n and r */
main ( )
{
    int  p, n, count;
    float r, si;

    for ( count = 1 ; count <= 3 ; count = count + 1 )
    {
        printf ( "Enter values of p, n, and r " );
        scanf ( "%d %d %f", &p, &n, &r );

        si = p * n * r / 100 ;
        printf ( "Simple Interest = Rs.%f\n", si );
    }
}

```

If this program is compared with the one written using **while**, it can be seen that the three steps—initialization, testing and incrementation—required for the loop construct have now been incorporated in the **for** statement.

Let us now examine how the **for** statement gets executed:

- When the **for** statement is executed for the first time, the value of **count** is set to an initial value 1.
- Now the condition **count** $\leq$ 3 is tested. Since **count** is 1 the condition is satisfied and the body of the loop is executed for the first time.
- Upon reaching the closing brace of **for**, control is sent back to the **for** statement, where the value of **count** gets incremented by 1.
- Again the test is performed to check whether the new value of **count** exceeds 3.
- If the value of **count** is still within the range 1 to 3, the statements within the braces of **for** are executed again.
- The body of the **for** loop continues to get executed till **count** doesn't exceed the final value 3.
- When **count** reaches the value 4 the control exits from the loop and is transferred to the statement (if any) immediately after the body of **for**.

The following figure would help in further clarifying the concept of execution of the **for** loop.

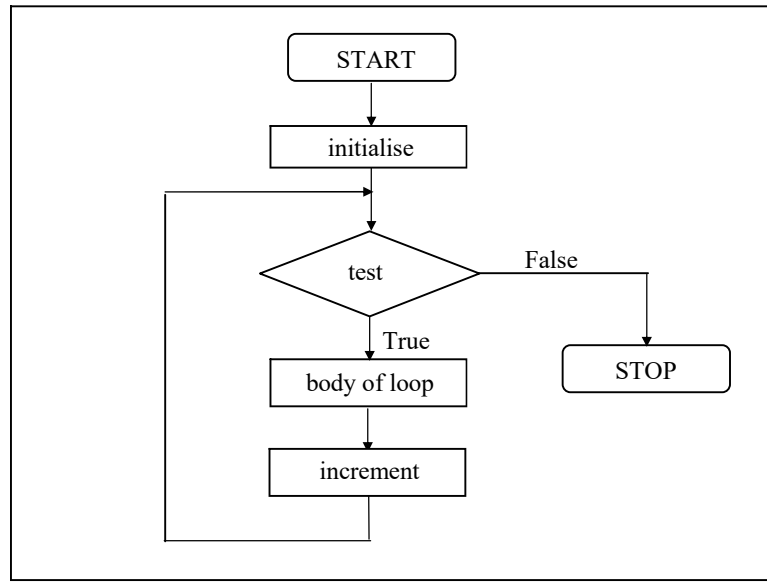


Figure 3.4

It is important to note that the initialization, testing and incrementation part of a **for** loop can be replaced by any valid expression. Thus the following **for** loops are perfectly ok.

```
for ( i = 10 ; i ; i -- )  
    printf ( "%d", i ) ;  
for ( i < 4 ; j = 5 ; j = 0 )  
    printf ( "%d", i ) ;  
for ( i = 1 ; i <= 10 ; printf ( "%d", i++ )  
    ;  
for ( scanf ( "%d", &i ) ; i <= 10 ; i++ )  
    printf ( "%d", i ) ;
```

Let us now write down the program to print numbers from 1 to 10 in different ways. This time we would use a **for** loop instead of a **while** loop.

```
(a) main( )
    {
        int i;
        for ( i = 1 ; i <= 10 ; i = i + 1 )
            printf ( "%d\n", i );
    }
```

Note that the initialisation, testing and incrementation of loop counter is done in the **for** statement itself. Instead of **i = i + 1**, the statements **i++** or **i += 1** can also be used.

Since there is only one statement in the body of the **for** loop, the pair of braces have been dropped. As with the **while**, the default scope of **for** is the immediately next statement after **for**.

```
(b) main( )
    {
        int i;
        for ( i = 1 ; i <= 10 ; )
        {
            printf ( "%d\n", i );
            i = i + 1 ;
        }
    }
```

Here, the incrementation is done within the body of the **for** loop and not in the **for** statement. Note that in spite of this the semicolon after the condition is necessary.

```
(c) main( )
    {
        int i = 1 ;
        for ( ; i <= 10 ; i = i + 1 )
            printf ( "%d\n", i );
    }
```

Here the initialisation is done in the declaration statement itself, but still the semicolon before the condition is necessary.

```
(d) main( )
{
    int i = 1 ;
    for ( ; i <= 10 ; )
    {
        printf ( "%d\n", i ) ;
        i = i + 1 ;
    }
}
```

Here, neither the initialisation, nor the incrementation is done in the **for** statement, but still the two semicolons are necessary.

```
(e) main( )
{
    int i ;
    for ( i = 0 ; i++ < 10 ; )
        printf ( "%d\n", i ) ;
}
```

Here, the comparison as well as the incrementation is done through the same statement, **i++ < 10**. Since the **++** operator comes after **i** firstly comparison is done, followed by incrementation. Note that it is necessary to initialize **i** to 0.

```
(f) main( )
{
    int i ;
    for ( i = 0 ; ++i <= 10 ; )
        printf ( "%d\n", i ) ;
}
```

Here, both, the comparison and the incrementation is done through the same statement, `++i <= 10`. Since `++` precedes `i` firstly incrementation is done, followed by comparison. Note that it is necessary to initialize `i` to 0.

Nesting of Loops

The way **if** statements can be nested, similarly **whiles** and **fors** can also be nested. To understand how nested loops work, look at the program given below:

```
/* Demonstration of nested loops */
main()
{
    int r, c, sum;
    for ( r = 1 ; r <= 3 ; r++ ) /* outer loop */
    {
        for ( c = 1 ; c <= 2 ; c++ ) /* inner loop */
        {
            sum = r + c;
            printf ( "r = %d c = %d sum = %d\n", r, c, sum );
        }
    }
}
```

When you run this program you will get the following output:

```
r = 1 c = 1 sum = 2
r = 1 c = 2 sum = 3
r = 2 c = 1 sum = 3
r = 2 c = 2 sum = 4
r = 3 c = 1 sum = 4
r = 3 c = 2 sum = 5
```

Here, for each value of **r** the inner loop is cycled through twice, with the variable **c** taking values from 1 to 2. The inner loop

terminates when the value of **c** exceeds 2, and the outer loop terminates when the value of **r** exceeds 3.

As you can see, the body of the outer **for** loop is indented, and the body of the inner **for** loop is further indented. These multiple indentations make the program easier to understand.

Instead of using two statements, one to calculate **sum** and another to print it out, we can compact this into one single statement by saying:

```
printf ( "r = %d c = %d sum = %d\n", r, c, r + c );
```

The way **for** loops have been nested here, similarly, two **while** loops can also be nested. Not only this, a **for** loop can occur within a **while** loop, or a **while** within a **for**.

Multiple Initialisations in the *for* Loop

The initialisation expression of the **for** loop can contain more than one statement separated by a comma. For example,

```
for ( i = 1, j = 2 ; j <= 10 ; j++ )
```

Multiple statements can also be used in the incrementation expression of **for** loop; i.e., you can increment (or decrement) two or more variables at the same time. However, only one expression is allowed in the test expression. This expression may contain several conditions linked together using logical operators.

Use of multiple statements in the initialisation expression also demonstrates why semicolons are used to separate the three expressions in the **for** loop. If commas had been used, they could not also have been used to separate multiple statements in the initialisation expression, without confusing the compiler.

The Odd Loop

The loops that we have used so far executed the statements within them a finite number of times. However, in real life programming one comes across a situation when it is not known beforehand how many times the statements in the loop are to be executed. This situation can be programmed as shown below:

```
/* Execution of a loop an unknown number of times */
main( )
{
    char another ;
    int num ;
    do
    {
        printf ( "Enter a number " ) ;
        scanf ( "%d", &num ) ;
        printf ( "square of %d is %d", num, num * num ) ;
        printf ( "\nWant to enter another number y/n " ) ;
        scanf ( " %c", &another ) ;
    } while ( another == 'y' ) ;
}
```

And here is the sample output...

```
Enter a number 5
square of 5 is 25
Want to enter another number y/n y
Enter a number 7
square of 7 is 49
Want to enter another number y/n n
```

In this program the **do-while** loop would keep getting executed till the user continues to answer y. The moment he answers n, the loop terminates, since the condition (**another == 'y'**) fails. Note that this loop ensures that statements within it are executed at least once even if **n** is supplied first time itself.

Though it is simpler to program such a requirement using a **do-while** loop, the same functionality if required, can also be accomplished using **for** and **while** loops as shown below:

```
/* odd loop using a for loop */
main( )
{
    char another = 'y';
    int num;
    for ( ; another == 'y'; )
    {
        printf ( "Enter a number " );
        scanf ( "%d", &num );
        printf ( "square of %d is %d", num, num * num );
        printf ( "\nWant to enter another number y/n " );
        scanf ( " %c", &another );
    }
}

/* odd loop using a while loop */
main( )
{
    char another = 'y';
    int num;

    while ( another == 'y' )
    {
        printf ( "Enter a number " );
        scanf ( "%d", &num );
        printf ( "square of %d is %d", num, num * num );
        printf ( "\nWant to enter another number y/n " );
        scanf ( " %c", &another );
    }
}
```

The *break* Statement

We often come across situations where we want to jump out of a loop instantly, without waiting to get back to the conditional test. The keyword **break** allows us to do this. When **break** is encountered inside any loop, control automatically passes to the first statement after the loop. A **break** is usually associated with an **if**. As an example, let's consider the following example.

Example: Write a program to determine whether a number is prime or not. A prime number is one, which is divisible only by 1 or itself.

All we have to do to test whether a number is prime or not, is to divide it successively by all numbers from 2 to one less than itself. If remainder of any of these divisions is zero, the number is not a prime. If no division yields a zero then the number is a prime number. Following program implements this logic.

```
main()
{
    int num, i;

    printf ( "Enter a number " );
    scanf ( "%d", &num );

    i = 2 ;
    while ( i <= num - 1 )
    {
        if ( num % i == 0 )
        {
            printf ( "Not a prime number" );
            break ;
        }
        i++ ;
    }
}
```

```
        if ( i == num )
            printf ( "Prime number" );
    }
```

In this program the moment **num % i** turns out to be zero, (i.e. **num** is exactly divisible by **i**) the message “Not a prime number” is printed and the control breaks out of the **while** loop. Why does the program require the **if** statement after the **while** loop at all? Well, there are two ways the control could have reached outside the **while** loop:

- (a) It jumped out because the number proved to be not a prime.
- (b) The loop came to an end because the value of **i** became equal to **num**.

When the loop terminates in the second case, it means that there was no number between 2 to **num - 1** that could exactly divide **num**. That is, **num** is indeed a prime. If this is true, the program should print out the message “Prime number”.

The keyword **break**, breaks the control only from the **while** in which it is placed. Consider the following program, which illustrates this fact.

```
main( )
{
    int i = 1 , j = 1 ;

    while ( i++ <= 100 )
    {
        while ( j++ <= 200 )
        {
            if ( j == 150 )
                break ;
            else
                printf ( "%d %d\n", i, j ) ;
        }
    }
```

```
    }  
}
```

In this program when **j** equals 150, **break** takes the control outside the inner **while** only, since it is placed inside the inner **while**.

The *continue* Statement

In some programming situations we want to take the control to the beginning of the loop, bypassing the statements inside the loop, which have not yet been executed. The keyword **continue** allows us to do this. When **continue** is encountered inside any loop, control automatically passes to the beginning of the loop.

A **continue** is usually associated with an **if**. As an example, let's consider the following program.

```
main()  
{  
    int i, j;  
  
    for ( i = 1 ; i <= 2 ; i++ )  
    {  
        for ( j = 1 ; j <= 2 ; j++ )  
        {  
            if ( i == j )  
                continue ;  
  
            printf ( "\n%d %d\n", i, j ) ;  
        }  
    }  
}
```

The output of the above program would be...

```
1 2  
2 1
```

Note that when the value of **i** equals that of **j**, the **continue** statement takes the control to the **for** loop (inner) bypassing rest of the statements pending execution in the **for** loop (inner).

The *do-while* Loop

The **do-while** loop looks like this:

```
do
{
    this ;
    and this ;
    and this ;
    and this ;
} while ( this condition is true ) ;
```

There is a minor difference between the working of **while** and **do-while** loops. This difference is the place where the condition is tested. The **while** tests the condition before executing any of the statements within the **while** loop. As against this, the **do-while** tests the condition after having executed the statements within the loop. Figure 3.5 would clarify the execution of **do-while** loop still further.

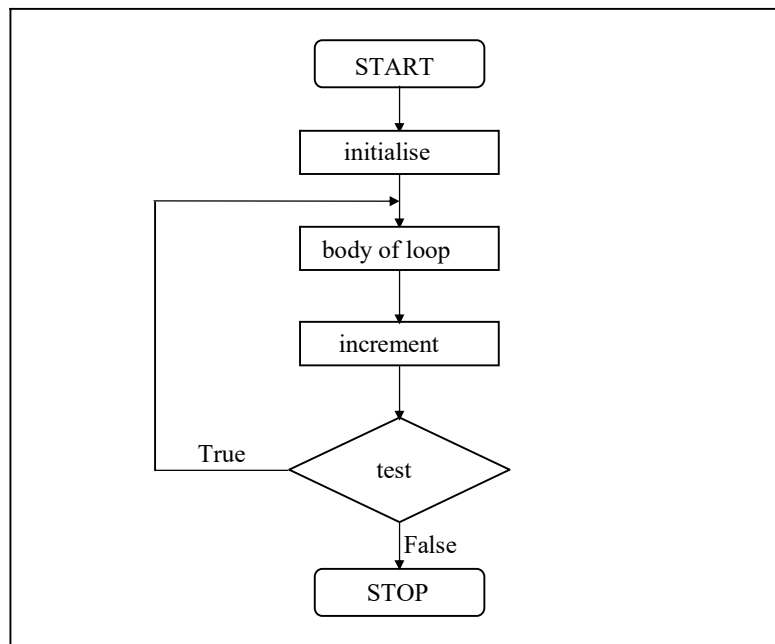


Figure 3.5

This means that **do-while** would execute its statements at least once, even if the condition fails for the first time. The **while**, on the other hand will not execute its statements if the condition fails for the first time. This difference is brought about more clearly by the following program.

```
main()  
{  
    while ( 4 < 1 )  
        printf ( "Hello there \n" );  
}
```

Here, since the condition fails the first time itself, the **printf()** will not get executed at all. Let's now write the same program using a **do-while** loop.

```
main( )
{
    do
    {
        printf ( "Hello there \n" );
    } while ( 4 < 1 );
}
```

In this program the **printf()** would be executed once, since first the body of the loop is executed and then the condition is tested.

There are some occasions when we want to execute a loop at least once no matter what. This is illustrated in the following example:

break and **continue** are used with **do-while** just as they would be in a **while** or a **for** loop. A **break** takes you out of the **do-while** bypassing the conditional test. A **continue** sends you straight to the test at the end of the loop.

Summary

- (a) The three type of loops available in C are **for**, **while**, and **do-while**.
- (b) A **break** statement takes the execution control out of the loop.
- (c) A **continue** statement skips the execution of the statements after it and takes the control to the beginning of the loop.
- (d) A **do-while** loop is used to ensure that the statements within the loop are executed at least once.
- (e) The ++ operator increments the operand by 1, whereas, the -- operator decrements it by 1.
- (f) The operators +=, -=, *=, /=, %= are compound assignment operators. They modify the value of the operand to the left of them.

Exercise

while Loop

[A] What would be the output of the following programs:

(a)

```
main( )
{
    int j;
    while ( j <= 10 )
    {
        printf ( "\n%d", j );
        j = j + 1 ;
    }
}
```

(b)

```
main( )
{
    int i = 1 ;
    while ( i <= 10 );
    {
        printf ( "\n%d", i );
    }
}
```

```
        i++ ;  
    }  
}
```

```
(c) main( )  
{  
    int j;  
    while ( j <= 10 )  
    {  
        printf ( "\n%d", j );  
        j = j + 1 ;  
    }  
}
```

```
(d) main( )  
{  
    int x = 1 ;  
    while ( x == 1 )  
    {  
        x = x - 1 ;  
        printf ( "\n%d", x );  
    }  
}
```

```
(e) main( )  
{  
    int x = 1 ;  
    while ( x == 1 )  
        x = x - 1 ;  
    printf ( "\n%d", x );  
}
```

```
(f) main( )  
{  
    char x ;
```

```
        while ( x = 0 ; x <= 255 ; x++ )
            printf ( "\nAscii value %d Character %c", x, x ) ;
    }

(g)  main( )
    {
        int x = 4, y, z ;
        y = --x ;
        z = x-- ;
        printf ( "\n%d %d %d", x, y, z ) ;
    }

(h)  main( )
    {
        int x = 4, y = 3, z ;

        z = x-- -y ;
        printf ( "\n%d %d %d", x, y, z ) ;
    }

(i)  main( )
    {
        while ( 'a' < 'b' )
            printf ( "\nmalyalam is a palindrome" ) ;
    }

(j)  main( )
    {
        int i = 10 ;
        while ( i = 20 )
            printf ( "\nA computer buff!" ) ;
    }

(k)  main( )
    {
        int i ;
        while ( i = 10 )
        {
```

```
        printf ( "\n%d", i );
        i = i + 1 ;
    }
}
```

```
(l)  main( )
    {
        float x = 1.1 ;
        while ( x == 1.1 )
        {
            printf ( "\n%f", x );
            x = x - 0.1 ;
        }
    }
```

```
(m)  main( )
    {
        while ( '1' < '2' )
            printf ( "\n\n while loop" );
    }
```

```
(n)  main( )
    {
        char x ;
        for ( x = 0 ; x <= 255 ; x++ )
            printf ( "\nAscii value %d Character %c", x, x );
    }
```

```
(o)  main( )
    {
        int x = 4, y = 0, z ;
        while ( x >= 0 )
        {
            x-- ;
            y++ ;
            if ( x == y )
```

```
        continue ;
    else
        printf ( "\n%d %d", x, y ) ;
    }
}
```

(p) `main( )`

```
{
    int x = 4, y = 0, z ;
    while ( x >= 0 )
    {
        if ( x == y )
            break ;
        else
            printf ( "\n%d %d", x, y ) ;
        x-- ;
        y++ ;
    }
}
```

[B] Attempt the following:

- (a) Write a program to calculate overtime pay of 10 employees. Overtime is paid at the rate of Rs. 12.00 per hour for every hour worked above 40 hours. Assume that employees do not work for fractional part of an hour.
- (b) Write a program to find the factorial value of any number entered through the keyboard.
- (c) Two numbers are entered through the keyboard. Write a program to find the value of one number raised to the power of another.
- (d) Write a program to print all the ASCII values and their equivalent characters using a **while** loop. The ASCII values vary from 0 to 255.

- (e) Write a program to print out all Armstrong numbers between 1 and 500. If sum of cubes of each digit of the number is equal to the number itself, then the number is called an Armstrong number. For example, $153 = (1 * 1 * 1) + (5 * 5 * 5) + (3 * 3 * 3)$
- (f) Write a program for a matchstick game being played between the computer and a user. Your program should ensure that the computer always wins. Rules for the game are as follows:
- There are 21 matchsticks.
 - The computer asks the player to pick 1, 2, 3, or 4 matchsticks.
 - After the person picks, the computer does its picking.
 - Whoever is forced to pick up the last matchstick loses the game.
- (g) Write a program to enter the numbers till the user wants and at the end it should display the count of positive, negative and zeros entered.
- (h) Write a program to find the octal equivalent of the entered number.
- (i) Write a program to find the range of a set of numbers. Range is the difference between the smallest and biggest number in the list.

for, break, continue, do-while

[C] What would be the output of the following programs:

- (a)

```
main( )
{
    int i = 0 ;
    for ( ; i ; )
```

```
        printf ( "\nHere is some mail for you" );  
    }
```

```
(b) main( )  
{  
    int i;  
    for ( i = 1 ; i <= 5 ; printf ( "\n%d", i ) );  
        i++;  
}
```

```
(c) main( )  
{  
    int i = 1, j = 1 ;  
    for ( ; ; )  
    {  
        if ( i > 5 )  
            break ;  
        else  
            j += i ;  
        printf ( "\n%d", j ) ;  
        i += j ;  
    }  
}
```

```
(d) main( )  
{  
    int i ;  
    for ( i = 1 ; i <= 5 ; printf ( "\n%c", 65 ) );  
        i++;  
}
```

[D] Answer the following:

(a) The three parts of the loop expression in the **for** loop are:

the i _____ expression
the t _____ expression
the i _____ expression

- (b) An expression contains relational operators, assignment operators, and arithmetic operators. In the absence of parentheses, they will be evaluated in which of the following order:
1. assignment, relational, arithmetic
 2. arithmetic, relational, assignment
 3. relational, arithmetic, assignment
 4. assignment, arithmetic, relational
- (c) The **break** statement is used to exit from:
1. an **if** statement
 2. a **for** loop
 3. a program
 4. the **main()** function
- (d) A **do-while** loop is useful when we want that the statements within the loop must be executed:
1. Only once
 2. At least once
 3. More than once
 4. None of the above
- (e) In what sequence the initialization, testing and execution of body is done in a do-while loop
1. Initialization, execution of body, testing
 2. Execution of body, initialization, testing
 3. Initialization, testing, execution of body
 4. None of the above
- (f) Which of the following is not an infinite loop.
1.

```
int i = 1;
while ( 1 )
{
    i++;
}
```
 2.

```
for(;;);
```

```
3. int True = 0, false ;
   while ( True )
   {
       False = 1 ;
   }

4. int y, x = 0 ;
   do
   {
       y = x ;
   } while ( x == 0 ) ;
```

- (g) Which of the following statement is used to take the control to the beginning of the loop?
1. exit
 2. break
 3. continue
 4. None of the above

[E] Attempt the following:

- (a) Write a program to print all prime numbers from 1 to 300.
(Hint: Use nested loops, **break** and **continue**)
- (b) Write a program to fill the entire screen with a smiling face.
The smiling face has an ASCII value 1.
- (c) Write a program to add first seven terms of the following series using a **for** loop:

$$\frac{1}{1!} + \frac{2}{2!} + \frac{3}{3!} + \dots$$

- (d) Write a program to generate all combinations of 1, 2 and 3 using **for** loop.
- (e) According to a study, the approximate level of intelligence of a person can be calculated using the following formula:

$$i = 2 + (y + 0.5x)$$

Write a program, which will produce a table of values of *i*, *y* and *x*, where *y* varies from 1 to 6, and, for each value of *y*, *x* varies from 5.5 to 12.5 in steps of 0.5.

- (f) Write a program to produce the following output:

```
A B C D E F G F E D C B A
A B C D E F   F E D C B A
A B C D E       E D C B A
A B C D           D C B A
A B C               C B A
A B                 B A
A                     A
```

- (g) Write a program to fill the entire screen with diamond and heart alternatively. The ASCII value for heart is 3 and that of diamond is 4.
- (h) Write a program to print the multiplication table of the number entered by the user. The table should get displayed in the following form.
- ```
29 * 1 = 29
29 * 2 = 58
...
```

- (i) Write a program to produce the following output:

```
 1
 2 3
 4 5 6
7 8 9 10
```